

认识 AI 从模型到工具

从 AI 的技术分层出发，理解程序员的沟通角色，再通过真实项目对话认识技术语言的价值。

14 个学习章节

概念关系

真实项目案例

技术语言与 AI 协作

01 两条主线：技术认知与效能工具

你就告诉我：一个不懂技术的人，是怎么看出过滤器写得有问题的？有同学说，AI 来了就不需要学技术了，我就纳闷了，到底要怎样跟 AI 沟通？这门系列课的名字叫作“AI 的技术认知和效能工具”。从名字就能看出来，它包括两块内容：一块是技术认知，一块是效能工具。为什么要讲这个系列课？因为这一年直播下来，直播间连麦的同学问这两块内容问得特别多。

这两块内容，也代表着现阶段同学们的一些重大疑问。在 AI 技术认知这一块，疑惑通常对应着焦虑和迷茫：不知道未来会怎么走，不知道学习还有没有意义，也不知道前端还能不能做。这些问题背后，是焦虑情绪和精神内耗。

很多时候，同学们不是被 AI 打败了，而是自己把自己打败了。打个夸张的比方：法医验尸没发现伤痕，人怎么没了？是吓死的。因此，第一块内容主要解决焦虑问题。我认为，焦虑背后的根本原因是技术认知不够。不懂当然会害怕。就像以前没有 AI 的时候，要使用一个自己没学过、没接触过的框架，你会不会害怕？肯定会。不懂就害怕，理解之后就不会那么害怕。我们会在技术认知这一部分，把目前 AI 涉及的技术格局讲清楚。

第二块是效能工具，主要围绕 AI 编程展开，解决工作实践中的问题。现在很多人都在用 AI 编程；在我的观察中，除了 AI 搞不定的部分，大量工作已经通过 AI 编程完成。当然也有特殊情况，例如军工或者涉密机构的内网环境，不方便使用这类工具，这些情况不在这里展开。

即使是同一款 AI 工具、同一种项目类型，不同的人用出来的效果也可能天壤之别。到底选什么工具、怎么使用？人在跟 AI 打交道时扮演什么角色？处理不同问题时，使用 AI 的方式是不是也要变化？效能工具这一部分会把这些问题讲清楚，我也会把自己的使用经验传递给大家。系列课主要就是这两块内容。



课程先提出技术认知与效能工具两条主线。

02 面对新名词，先看技术结构

先看 AI 技术认知。它不仅解决焦虑，也解决技术知识体系的问题。很多同学对 AI 一知半解，知识来源几乎都是自媒体：每天都是“炸裂”“震惊”“颠覆”。如果对整个技术体系的认知很模糊，就很容易被带节奏；一出现新东西，就觉得不得了，天又塌了。

从 2023 年到课程讲解的这个阶段，AI 已经发展了一段时间，天真的塌了吗？并没有。但是在短视频环境里，好像已经塌过无数次。根本问题是，脑子里没有形成正确的技术体系。随便列举一些 AI 领域的名词，可能就会看得头疼。我这里只是随手列了一部分，还没有列完；未来还会冒出更多名词和概念。你不必先把它们逐个背下来。

看待 AI，不要只从技术名词出发，应该从技术结构出发。这样它就会变得简单得多。课程把 AI 的技术结构分成三层：裸模型、模型服务、AI 应用。后续内容也会按这三层来讲。最底层是裸模型，也就是平时听到的各种大模型。



讲者列举了大量 AI 技术名词，说明仅记忆名词难以建立清晰的知识体系。

03 第一层：裸模型

例如 GPT-5.2、国产的 Kimi、智谱的模型，以及 Gemini 等，都属于这里所说的裸模型。这个层面除了具体模型的名称，还包括注意力机制、KV-Cache、上下文窗口等技术概念。这些都归到裸模型这一层，后面的课程会详细讲解。



裸模型一行列出模型名称、注意力机制、KV-Cache 和上下文窗口。

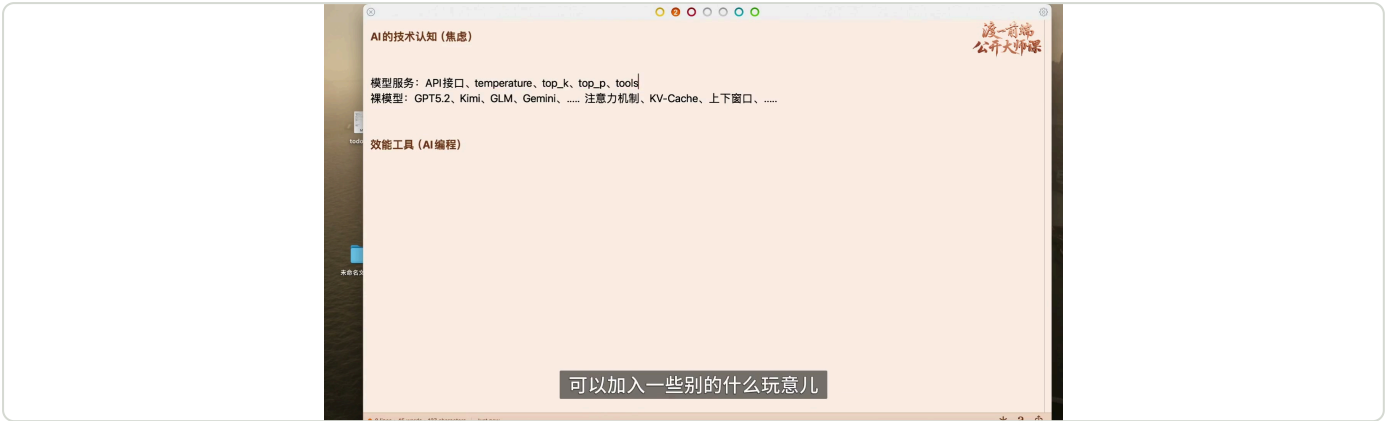
画面中的细节

- 画面中出现的模型名称：GPT5.2、Kimi、GLM、Gemini。
- 模型相关术语：注意力机制、KV-Cache、上下文窗口。

04 第二层：模型服务

在裸模型的基础上，可以再做一层封装，这些封装往往由模型服务商提供，我们把它叫作模型服务。有些同学已经接触得比较多：制作模型的公司，会提供一个 API 接口。比如课程以 OpenAI 提供 GPT 模型服务为例，说明模型提供商如何把模型能力开放出来。

这个 API 是一个远程接口，程序可以按照它的协议连接、提出问题，再获得回复。它是在裸模型基础上提供的封装，通常以 API 接口的形式表现出来。模型服务层也有自己的概念，例如 temperature、top_k、top_p、tools 等。不要把这些参数名和模型内部机制混在一起看，它们属于这里讨论的模型服务层。



模型服务层列出 API 接口及 temperature、top_k、top_p、tools 等。

画面中的细节

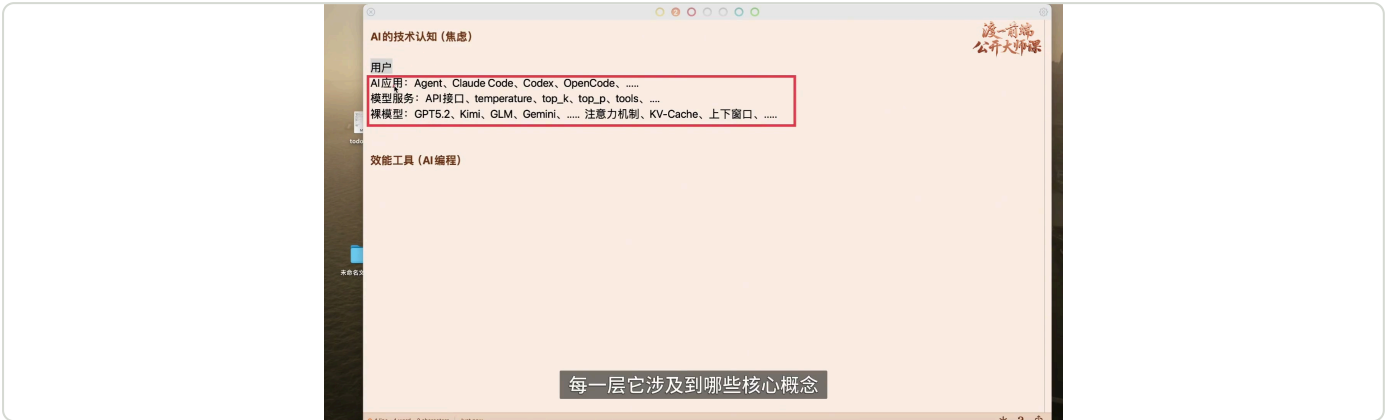
- API 参数按原板书保留为 temperature、top_k、top_p、tools。

05 第三层：AI 应用与学习路径

再往上一层，是对模型服务的进一步封装：调用这些接口，做出具体的 AI 应用。Agent、Claude Code、Codex、OpenCode，以及我们使用的各种 AI 编程应用，都放在这一层讨论。AI 应用的上面，就是直接使用它们的用户。大体上就是这样一个格局。

理解了这个格局，再来看效能工具属于哪一层：用来进行 AI 编程的工具，属于直接提供给用户使用的应用层。如果有一天你要开发一个 AI 应用，重点就要理解模型服务层，同时理解 AI 应用中常见的开发手段；应用层也会引出自己的一些概念，因此这两层都需要关注。

如果要做模型微调，就会涉及裸模型层，因为要改动模型。后续课程会先讲裸模型，再讲模型服务，最后讲 AI 应用，把每层的核心概念和使用方式讲清楚。理解这个结构之后，即使又出现许多新概念，也可以判断它属于哪一层、在那一层起什么作用，而不必一遇到新名词就害怕。



完整板书：用户、AI 应用、模型服务、裸模型。

画面中的细节

- 应用层示例：Agent、Claude Code、Codex、OpenCode。
- 用户位于应用层上方；使用工具、开发应用、微调模型对应不同关注层次。

06 认知差异，决定使用差异

这个道理和过去学习前端、后端没有本质区别。同样学习 Vue、React 等框架，为什么有的人薪资高，有的人薪资低？为什么有的人做得好，有的人做得差，甚至做不出来？认知上的差异，会造成结果上的差异。到了 AI 时代，这个逻辑没有变：越理解 AI，越能把它用好；概念越模糊，知识盲区越多，使用效果就越差。

认知模糊还容易让人反复内耗、陷入焦虑。再举一个例子：如果你已经能把 Vue 的源码自己写出来，还会对这个框架那么焦虑吗？别人提问你不怕，工作中遇到场景也知道怎样处理。这想说明的是，恐惧来源于未知。接下来讨论第二部分：如何使用 AI 编程，也就是 AI 效能工具。



通过框架学习的类比，说明理解深度与使用效果的关系。

07 程序员是需求与运行时之间的桥梁

这一部分的核心，是程序员在 AI 时代应该承担什么角色。我希望在课程中把它讲清楚，后续也会详细介绍具体的 AI 编程工具和使用方法。先简单看一下：现在的程序员角色和过去确实有变化，但有没有根本性的变化？我的判断是没有。

过去没有 AI 的时候，程序员承担什么角色？如果回答“编程”，还可以继续问：编程本质上是什么角色？它其实是一种沟通角色，连接现实世界和计算机运行时。运行时就是运行代码的环境，例如浏览器、服务器。程序员就像两者之间的一座桥梁。

现实世界中的需求，可能来自产品经理、老板，也可能是你自己想到的。需求传到程序员这里，经过理解、加工和转换，变成计算机运行时能够理解的语言，也就是代码，再交给运行环境执行。运行得到的结果会反馈给程序员。

程序员根据结果判断：有没有满足现实世界的需求？没有满足，就继续沟通和调整。现在这个角色并没有消失，只是在沟通者和运行时之间，多了一个 AI。程序员这个环节仍然重要。为什么不能简单地让产品经理或者老板替代这一环节？因为跟 AI 沟通，虽然看起来用的是自然语言，却不完全日常自然语言。



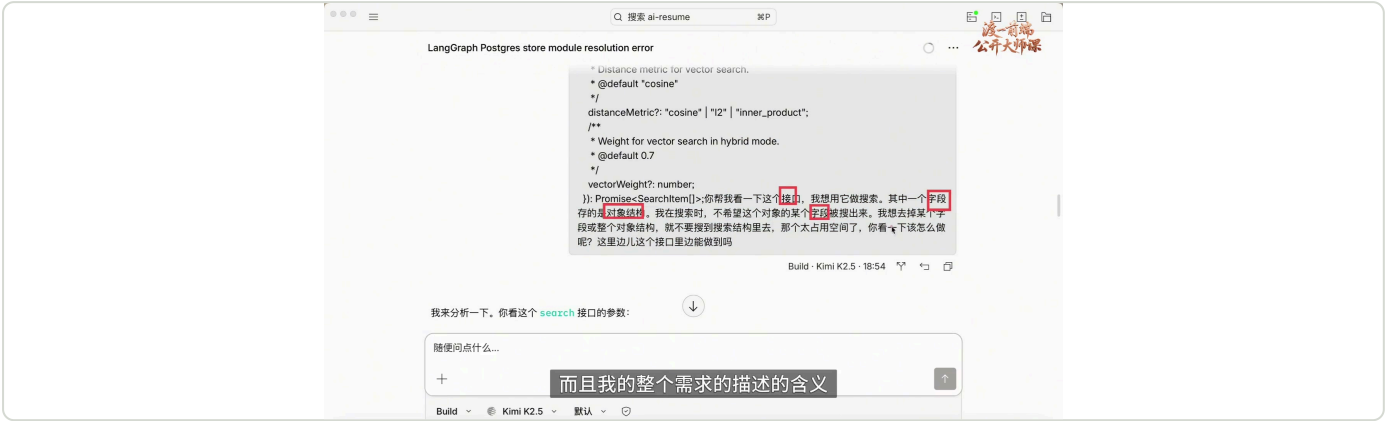
现实世界、沟通者和计算机运行时构成需求实现与反馈链路。

08 案例一：用技术语言分析搜索接口

我们需要的是技术语言。怎么理解？先看一个例子：我把一个函数定义贴给 AI。这个函数参数很多，我不想逐项去看，就让它帮我分析接口。我想用这个接口实现一些功能，需要确认现有函数能不能满足需求。表面上，我确实是在用自然语言提问。

但这里不是完全脱离技术背景的日常对话。问题中涉及接口、字段、对象结构，整个需求描述也都与技术有关。完全不懂技术的人，可能根本不知道这些东西存在，又怎么把问题说出来？画面中的问题具体是：搜索接口中的某个字段存放了对象结构，搜索时不希望把某个子字段或者整个对象结构带入搜索结果，因为太占空间；需要判断这个接口能否做到、应该怎样做。

更不用说前面的函数定义本来就是从代码里复制出来的。如果看不懂技术语言，不知道这段代码是什么，又怎么提出这个问题？提出问题的项目背景可能很复杂，这里不展开。但在项目中总会遇到需要细化、需要追问具体技术问题的时候，不懂技术就难以把问题说清楚。下面再看另一个例子：我用语音输入向 AI 反馈问题，文字识别有些零乱，但并不影响这个例子要说明的事。



搜索接口提问：如何避免把某个字段或对象结构带入搜索结果。

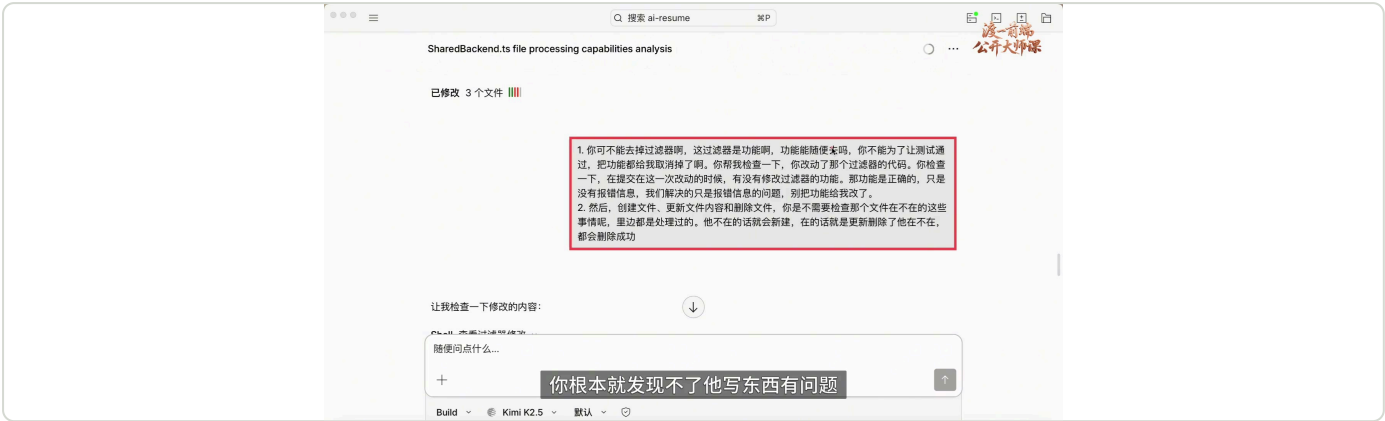
画面中的细节

- 示例来自 ai-resume 项目，界面标题为“LangGraph Postgres store module resolution error”。
- 函数定义中可见 distanceMetric 的候选值 cosine、l2、inner_product，以及 vectorWeight。
- 提问要求判断现有 search 接口能否控制返回字段，并没有在本课展示完整解决方案。

09 案例二：测试通过，功能仍可能被改坏

我发现 AI 写的东西有问题。它可能告诉你问题已经解决、测试也通过了，看起来都没问题，但我在它修改的过程中就发现它动了我的过滤器。过滤器是这个项目中的一个具体功能点，这里不用先解释这个功能本身。关键是：如果不懂技术，怎么发现过滤器被改得有问题？测试通过，并不能代替对功能是否被破坏的判断。

有同学说“AI 来了，就不需要学技术了”，我就纳闷：你到底怎么跟 AI 沟通？现在当然很多人用过 AI，但为什么用过之后还会得出这样的结论？再看一个例子：在 AI 编程中，有一块处理比较复杂，我希望它把相关内容写成一篇文档。可是 AI 不一定了解项目中使用的库，因为模型的训练语料有时效性。



过滤器属于功能，不能为了测试通过而直接移除或改变其行为。

画面中的细节

- 第一点：检查改过的过滤器代码，只解决报错信息问题，不要改掉原有功能。
- 第二点：创建、更新、删除文件的存在性处理，项目内部已有约定；不能重复加入不符合约定的判断。
- 这些要求属于该项目的语义，不能直接推广为所有文件系统都适用的规则。

10 案例三：理解代码，再沉淀文档

比如，假设某个模型是在 2025 年训练出来的，它可能不了解之后出现的一些技术，这些内容需要你自己熟悉。对于复杂部分，如果不懂技术，首先手写就可能搞不定。我处理好之后，希望 AI 把相关内容整理成文档，就需要大致告诉它这一块在做什么，让它阅读代码、分析逻辑，再把内容沉淀下来。

在这个例子里，我告诉它：这里是一个虚拟文件系统，数据通过 store 存在数据库里。我没有逐一告诉它数据库用的是什
么，因为它可以从代码分析；但这种关键点仍然需要提示。不懂技术，就很难给出合适的提示。再看一个提问：AI 返回了一种看起来奇怪的对象结构，我产生了疑问，追问为什么要这样返回。



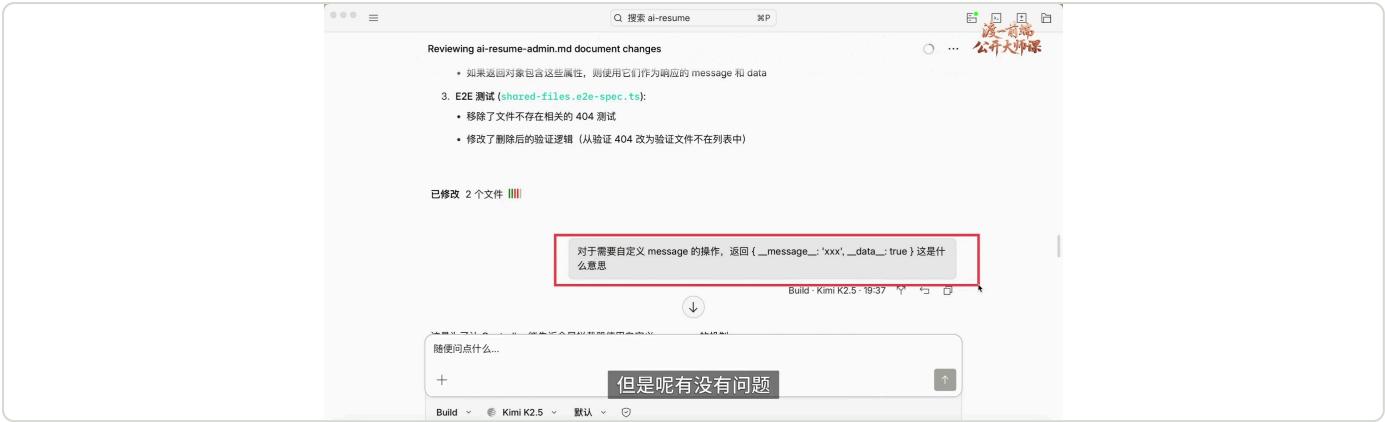
RouteBackend.ts 的虚拟文件系统说明与文档整理请求。

画面中的细节

- 请求明确指向 servers/ai-resume-server/src/common/resume-agent/components/backend/RouteBackend.ts。
- 讲者提示：生成虚拟文件系统，数据通过 store 存入数据库，请先总结这些操作。
- AI 回复画面中提到 CompositeBackend、UserIsolatedBackend 和 SharedBackend。本课没有进一步讲解完整实现。

11 案例四：发现疑问，追问返回结构

先不用急着判断它到底写错没有。在这个具体例子里，它写得没有问题，但你怎么知道它没有问题？仍然需要提出疑问、追问解释。如果完全不懂技术，可能连应该在哪里产生疑问都不知道。说来说去，我还是想不明白：为什么有人认为 AI 出来了就不用学技术？不学技术，就缺少技术语言，也就很难进行有效的技术沟通。



追问返回对象中 __message__、__data__ 的含义。

画面中的细节

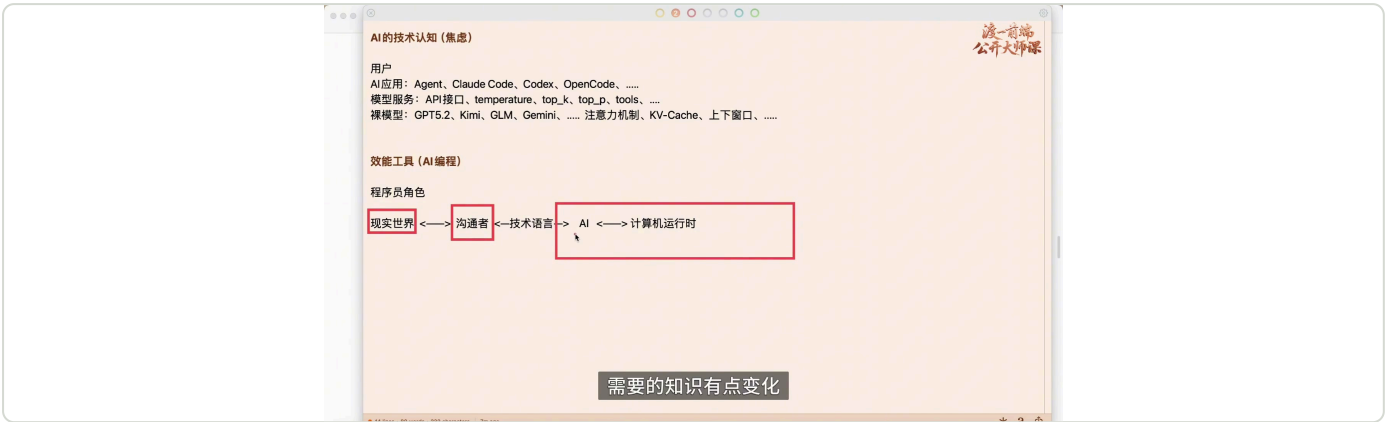
- 画面中的返回示例：{ '__message__': 'xxx', '__data__': true }。
- 上方说明：返回对象含这些属性时，用作响应的 message 和 data。
- 画面同时提到 E2E 测试 shared-files.e2e-spec.ts：移除文件不存在的 404 测试，删除后的验证改为检查文件不在列表中。这是项目案例细节。

12 商业应用需要持续的技术沟通

当然，如果只是让 AI 做一个俄罗斯方块或者贪吃蛇，这些简单例子另说。但实际工作要开发的是商业应用：代码少则几万行，多则几十万行，甚至上百万行。真的能只说一句话，就把剩下的事情全部交给 AI 做完吗？如果一句话做不完，就还需要持续沟通；持续沟通，就离不开技术语言。

因此，程序员的定位仍然是现实世界与技术世界之间的桥梁。需要的知识有变化，但根本逻辑没有变。既然要用技术语言跟 AI 沟通，就仍然要懂技术；理解得越深、越广、越多，沟通才越有基础。

技术理解越充分，给 AI 的指令、方向和步骤，就越有可能精确、恰当。我还用一个现象来反问：这些年 AI 不断发展，网上不断说“炸裂”“颠覆”，但为什么我观察到的招聘要求并没有降低？如果说 AI 已经把程序员需要承担的事情都替代了，企业对人的要求是不是应该相应降低？



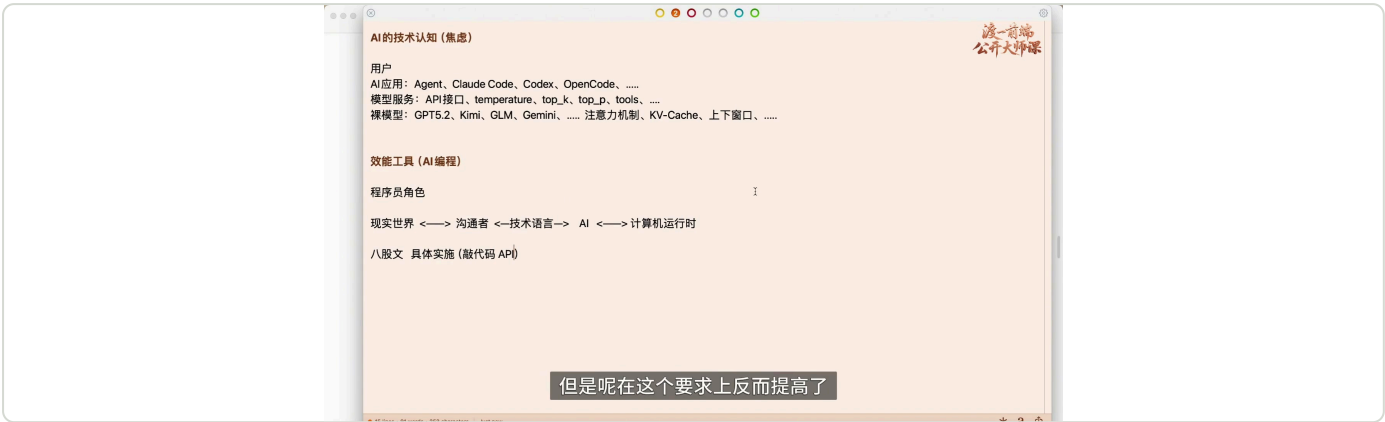
AI 加入协作链路后，人与 AI 之间仍通过技术语言沟通。

13 能力要求的重心正在变化

假设一家企业认为 AI 已经强到不需要程序员做多少事情了，那么它似乎就可以降低要求、降低薪资，比如只给三千、五千，让人只学会聊天和使用工具即可。但在我的观察中，事情没有这样发生，要求反而变高了。这是我用来说明技术沟通重要性的反问：要和一个能力范围很广的工具协作，你也需要有足够的知识来判断和引导它。

另一方面，既然不再那么依赖程序员自己逐行写代码，哪些层面的要求相对降低了？我指的是八股文、具体实施细节，以及某些 API、函数、对象到底怎样使用等层面。简单说，就是具体敲代码的那一部分。这不等于所有技术要求都消失了。

相对提高的，是思维、认知，以及知识体系的广度和深度。希望听完这节课之后，不要再简单地用“AI 来了就淘汰程序员”来概括问题。如果对这件事仍有疑问，可以继续听这门系列课程，结合后续的技术认知与使用实践去理解。这里关于招聘和能力要求的讨论，是课程讲者的观察与论证。



板书区分具体实施、API 使用等细节与更广泛的技术认知。

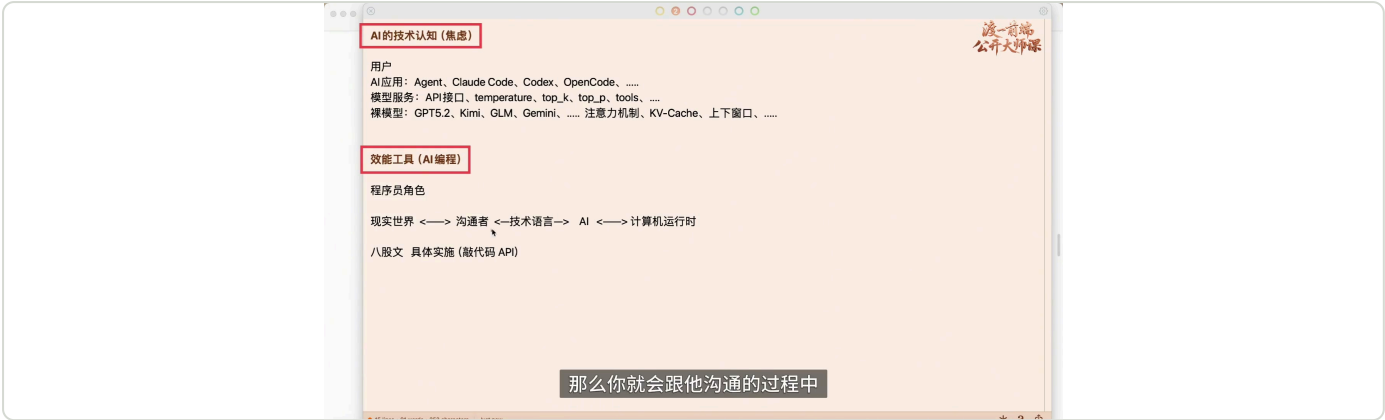
画面中的细节

- “招聘要求没有降低”及其原因，是课程讲者的观察与推论，没有在原视频中提供统计数据。

14 让认知与工具使用相互促进

在效能工具、AI 编程这一部分，我们主要会处理：怎样选择适合自己或者适合项目的 AI 工具，以及用什么方式跟 AI 沟通效果更好。两条主线也相互关联：前面的技术认知越深刻、越准确，后面的 AI 编程工具就越容易用好。

你知道 AI 擅长什么、不擅长什么、哪些地方容易出问题，就能在沟通和使用中尽量规避这些问题。这就是整个系列课要讲的格局。接下来会先从技术认知开始，一步一步展开。



两条主线最终回到同一目标：理解 AI，并更好地使用它。